

《人工智能》Project 1：搜索

- 课程号：CS50020；时间：2026.10

项目介绍

本课程项目分为课上实验和课下作业两部分：课上实验部分将完成与搜索算法相关的核心代码；课下作业为基于核心代码的扩展实验。本次课程项目以勇士斗恶龙中的搜索问题为例展开。

重要时间点

内容	开始时间	截止时间	提交方式	提交内容
课前热身	9.30	10.8 12:00	Elearning 提交	提交截图
课上实验	10.8 13:30	10.8 16:10	每组当堂验收	讲解算法及结果
课后作业	10.8	10.18 23:59	Elearning 提交	报告 + 代码

作业要求

- 1-2 人组队完成；
- 发布和提交平台：项目的发布和提交都通过 elearning 平台完成。
- 课后作业提交要求：
 - 每组提交一份 PDF 报告，报告需包含：（1）课上实验核心代码算法逻辑的简要回顾；（2）扩展实验完成情况及自动评价分数；（3）具体人员信息和分工。命名格式为提交同学的 学号-姓名-PJ1 报告.pdf；
 - 每组提交一份代码附件压缩包，包含实现代码后的完整实验文件夹。命名格式为与报告命名相同的同学的 学号-姓名-PJ1 代码.zip。
- 课程项目均需要使用 Python 编程，请提前安装好 Python 环境（推荐使用 Anaconda）和集成开发环境（VS Code、PyCharm、Spyder 等）；
- 课程项目的代码实现严禁抄袭，相似度过高的作业提交将记为无效成绩。
- 评分设置：课上核心代码实现当堂验收（30%）+ 课下作业报告提交（70%）。课上实验逾期未能验收时，仅验收最终报告内的课上实验介绍，并最多获得 $30\% \times 60\% = 18\%$ 的课上分数。

实验环境介绍

在本课程项目中，勇士智能体将在迷宫世界中寻找路径，到达特定的位置并有效地收集金币。勇士需要在避免被恶龙捉到的前提下觅食。本课程项目的目标是构建通用的搜索算法并将其应用于勇士场景。所有需要使用的代码文件均包含在 `pj1-search.zip` 中。在实现过程中，只需要关注其中的几个文件，可以忽视其他支持文件：

需要编辑的文件

文件	说明
<code>search.py</code>	用于存放所有搜索算法的代码文件

文件	说明
<code>searchAgents.py</code>	用于存放所有基于搜索的智能体的代码文件
<code>multiAgents.py</code>	用于存放多智能体

可以参考的文件

文件	说明
<code>hero.py</code>	勇士主文件，定义了勇士的状态（ <code>GameState</code> ）
<code>game.py</code>	游戏逻辑文件，定义了 <code>AgentState</code> 、 <code>Agent</code> 、 <code>Direction</code> 和 <code>Grid</code> 等类型
<code>util.py</code>	用于实现搜索算法的数据结构

Part 1. 课前热身内容

Q0. 上手体验勇士挑战恶龙游戏

最终目标：熟悉游戏运行方式、地图参数和智能体入口。（本题无需修改代码；运行 `hero.py` 体验游戏。）

下载并解压 `PJ1-搜索.zip` 文件，进入文件夹目录后，在终端中输入以下命令来体验勇士挑战恶龙游戏，并通过键盘方向键进行控制：

```
python hero.py
```

通过 `--layout` 参数和 `--hero` 参数指定迷宫地图和勇士智能体。通过输入 `multiAgents.py` 中的 `Agent` 类名指定要使用的勇士智能体。

如下面这行命令指定了一个只向西移动的智能体：

```
python hero.py --layout testMaze --hero GoWestAgent
```

在一些情况下，这种简单的智能体无法解决复杂的迷宫问题，可以通过 `Ctrl+C` 终止程序：

```
python hero.py --layout tinyMaze --hero GoWestAgent
```

我们也可以切换稍微复杂一些的智能体：

```
python hero.py --hero ReflexAgent
```

可以通过以下命令查看所有可用参数和说明：

```
python hero.py -h
```

课前热身提交内容

解决以下问题：（一行代码或多行代码）

问题 1：给定当前游戏状态 `gameState`，该如何获取金币（`coin`）的坐标列表？（可以使用 `type` 确定变量的类型，然后寻找对应类的相关方法。）

问题 2：给定当前游戏状态 `gameState`，如何获得第二条恶龙的位置？

Part 2. 课上实验内容

Q1. 广度优先搜索 BFS

需要完成： `search.py` 中的 `breadthFirstSearch` 函数。

最终目标： 实现图搜索版 BFS，返回从起点到目标的合法动作列表，并避免重复扩展状态。

首先，使用在 `searchAgents.py` 中定义好的 `SearchAgent` 测试整体实验环境：

```
python hero.py -l tinyMaze -p SearchAgent -a fn=tinyMazeSearch
```

Q1 至 Q2 将专注于课程中介绍的几种搜索算法的核心代码实现，搜索算法的伪代码可以参考课程视频。这些算法都具有相似的标准搜索流程，仅在对搜索边缘中节点的弹出逻辑存在区别。

注意：搜索节点不仅必须包含状态，还必须包含重构到达该状态的路径（规划）所需的信息。所有搜索函数都需要返回一个操作列表，将智能体从开始状态引导到目标状态。这些动作都必须是合法的（即有效的方向，不能穿墙）。本项目的数据结构需要使用 `util.py` 中提供的 `Stack`、`Queue` 和 `PriorityQueue` 等，这些数据结构的实现具有与自动评分器兼容所需的特定属性。

本题要求在 `search.py` 中的 `breadthFirstSearch` 函数中实现广度优先搜索（BFS）算法。为了使算法完整，本题要求编写 BFS 的图搜索版本，以避免扩展任何已经访问过的状态。

实现 BFS 算法后，可以通过以下命令测试搜索算法是否按预期找到搜索问题的解：

```
python hero.py -l tinyMaze -p SearchAgent -a fn=bfs
python hero.py -l mediumMaze -p SearchAgent -a fn=bfs
python hero.py -l bigMaze -z .5 -p SearchAgent -a fn=bfs
```

勇士界面将显示被探索的状态，以及它们被探索的顺序（较亮的红色表示更早的探索）。

思考： BFS 是否找到了成本最低的解决方案？如果没有，请检查代码实现。

补充： 如果本题 BFS 的代码实现是规范且通用的，那么该代码同样适用于 8 数码的搜索问题，可以直接运行以下命令执行：

```
python eightpuzzle.py
```

成绩评测：

运行以下命令，查看代码实现是否通过了自动评分器的所有测试用例：

```
python autograder.py -q q1
```

Q2. A* 搜索

需要完成： `search.py` 中的 `astarSearch` 函数。

最终目标： 实现通用的 A* 图搜索，使用传入的启发式函数返回到达目标的合法动作列表。

本题要求在 `search.py` 中的空函数 `astarSearch` 中实现 A* 图搜索。A* 搜索函数以一个启发式函数作为参数。启发式函数接收两个参数：搜索问题中的状态（主要参数）和搜索问题本身（用于参考信息）。`search.py` 中的 `nullHeuristic` 是一个简单的平凡启发式函数例子，它只返回代价 0，此时 A* 搜索相当于一致代价搜索。

本题可以使用预先定义好的曼哈顿距离启发式（在 `searchAgents.py` 中已经以 `manhattanHeuristic` 函数实现）来测试 A* 搜索，以解决在迷宫中寻找固定金币路径的搜索问题。

```
python hero.py -l bigMaze -z .5 -p SearchAgent -a
fn=astar,heuristic=manhattanHeuristic
```

通过运行以上命令可以发现，A* 搜索找到最优解决方案的速度略快于一致代价搜索。

成绩评测：

运行以下命令，查看代码实现是否通过了自动评分器的所有测试用例：

```
python autograder.py -q q2
```

Q3. 极小极大算法

需要完成： `multiAgents.py` 中的 `MinimaxAgent.getAction` 方法。

最终目标： 实现支持任意数量恶龙和任意搜索深度的极小极大决策，并返回 Hero 的动作。

这一题需要你在 `multiAgents.py` 提供的 `MinimaxAgent` 类中编写一个对抗搜索智能体。并且这个极小极大智能体需要处理任意数量的恶龙。特别地，极小极大树在每一个最大层（Hero 层）后面都会有多个最小层（每条恶龙各一层）。

代码还需要实现将游戏树扩展到任意深度。使用提供的 `self.evaluationFunction` 为极小极大树的叶子打分，默认情况下它使用 `scoreEvaluationFunction`。`MinimaxAgent` 继承自 `MultiAgentSearchAgent`，因此可以访问 `self.depth` 和 `self.evaluationFunction`。请确保你实现的极小极大算法使用了这两个变量，因为这些变量会根据命令行选项自动设置。

小贴士：

1. 正确的极小极大算法实现会导致勇士在某些测试中输掉——这是正常的，并且能够通过测试。
2. 一个搜索层被视为一次勇士的移动以及所有恶龙的移动，因此深度 2 的搜索将涉及勇士和每条恶龙移动两次。

在角落迷宫 (Corners) 中, 每个角落有一个点或金币, 一共有四个角落。该搜索问题的设定是找到穿过迷宫的所有四个角落的最短路径。注意, 对于一些迷宫 (如 `tinyCorners`), 最短路径并不总是先到达最近的角落。提示: 通过 `tinyCorners` 的最短路径需要 28 步。

注意: 确保在处理 Q4 之前完成 Q1。Q1 中实现的 BFS 负责搜索并找到最短路径; Q4 不需要重新实现搜索算法, 而是需要将“经过所有角落”建模为一个搜索问题。具体来说, 需要为 `CornersProblem` 设计合适的搜索状态, 使其既能记录 Hero 当前的位置, 也能记录已经访问过的角落, 并通过

`getStartState`、`isGoalState` 和 `getSuccessors` 向 BFS 提供搜索所需的信息。完成这些内容后, Q1 中已有的 BFS 就可以搜索出经过全部四个角落的最短路径。

本题要求在 `searchAgents.py` 中实现 `CornersProblem` 搜索问题。本题需要编写一种对搜索状态的表示, 它编码了目标检测所需的所有信息。实现代码后, 搜索智能体可以解决以下命令指定的搜索问题:

```
python hero.py -l tinyCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

```
python hero.py -l mediumCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

为了实现全部指标要求, 本题还需要定义一个抽象的搜索状态表示, 不编码无关信息 (如恶龙的位置、额外金币的位置等)。特别是不要将勇士的游戏状态 (`GameState`) 用作搜索状态, 否则代码将会变得非常慢。`CornersProblem` 类的实例表示本题的整个搜索问题, 而不是某个特定状态。特定状态由本题要求编写的函数返回, 并使用相应的数据结构 (如 `tuple`、`set` 等) 表示。

此外, 程序运行时会同时存在许多状态, 它们都在搜索算法的队列中, 而且应该彼此独立。换句话说, 整个 `CornersProblem` 对象不应该只有一个状态; 本题实现的类应该能够生成许多不同的状态并提供给搜索算法。提示: 在实现中只需要引用游戏状态中的勇士起始位置和四个角的位置。在编写 `getSuccessors` 方法时, 确保将子节点添加到 `successors` 列表中, 并将成本设为 1。

本题实现的 `breadthFirstSearch` 在 `mediumCorners` 上扩展大约 2000 个节点。启发式方法可以减少所需的搜索量, 这将在 Q5 中具体讨论。

成绩评测:

运行以下命令, 查看代码实现是否通过了自动评分器的所有测试用例:

```
python autograder.py -q q4
```

Q5. 角落问题的启发式函数设计

需要完成: `searchAgents.py` 中的 `cornersHeuristic` 函数。

最终目标: 为 `CornersProblem` 提供非负、非平凡且一致的启发式函数, 减少 A* 的节点扩展并保持最优性。

注意: 请确保在完成 Q5 之前完成 Q2, 因为 Q5 建立在 Q2 答案的基础上。

本题要求为 `CornersProblem` 实现一个非平凡且一致的 `cornersHeuristic` 启发式函数:

```
python hero.py -l mediumCorners -p AStarCornersAgent -z 0.5
```

一致的启发式函数: 在 A* 图搜索中, 启发式函数必须满足一致性才能保证搜索到最优解。

非平凡的启发式函数: 平凡启发式包括处处返回代价 0 的启发式函数 (即 UCS), 以及计算实际代价的启发式函数。前者不会节省任何时间, 而后者将使自动评分器超时。因此, 本题需要一个能够减少总计算时间的有效启发式函数。

评分标准： 本题编写的启发式函数必须是非平凡、非负且一致的启发式函数。确保启发式函数在抵达任意目标状态时都返回 0，并且永远不会返回负值。根据启发式函数扩展的节点数，成绩评级如下：

扩展节点数	分数
超过 2000	0/3
不超过 2000	1/3
不超过 1600	2/3
不超过 1200	3/3

成绩评测：

运行以下命令，查看代码实现是否通过了自动评分器的所有测试用例：

```
python autograder.py -q q5
```

Q6. 吃完所有金币

需要完成： `searchAgents.py` 中的 `coinHeuristic` 函数；可选 `jev` 扩展另见 `jev/`。

最终目标： 为 `CoinSearchProblem` 提供有效且一致的启发式函数，使 A* 更高效地找到收集全部金币的路径。

注意：请确保在处理 Q6 之前完成 Q2，因为 Q6 是在回答 Q2 的基础上提出的。

本题旨在解决一个困难的搜索问题：用尽可能少的步骤吃掉所有金币。为此，实验环境将该问题形式化定义为 `searchAgents.py` 中的 `CoinSearchProblem`（已实现）。该问题的解是收集全部金币的路径。对于目前的项目，解决方案不考虑恶龙或宝剑，而只取决于迷宫墙壁、金币和勇士的位置。正确实现通用搜索方法后，使用 `nullHeuristic`（相当于一致代价搜索）的 A* 搜索可以快速找到 `testSearch` 的最优解决方案（总代价为 7）：

```
python hero.py -l testSearch -p AStarCoinSearchAgent
```

使用 `nullHeuristic` 的 A* 搜索相当于 UCS，这种搜索算法在该问题中的效率很低。对于 `tinySearch`，UCS 在扩展 5057 个搜索节点后耗时 2.5 秒，找到了长度为 27 的路径。

本题要求在 `searchAgents.py` 中填写 `coinHeuristic`，为金币搜索问题提供满足一致性的启发式函数，并在 `trickySearch` 迷宫上测试智能体：

```
python hero.py -l trickySearch -p AStarCoinSearchAgent
```

UCS 下的智能体在大约 13 秒内找到了最优解决方案，探索了超过 16,000 个节点。

评分标准： 根据启发式扩展的节点数量，成绩分级如下：

扩展节点数	分数
超过 15000	0/4

扩展节点数	分数
不超过 15000	1/4
不超过 12000	2/4
不超过 9000	3/4
不超过 7000	4/4

成绩评测：

运行以下命令，查看代码实现是否通过了自动评分器的所有测试用例：

```
python autograder.py -q q6
```

可选扩展：使用 Jev 作为 Q6 启发式（不计入成绩）

完成并通过原始 `coinHeuristic` 后，可以选择使用 Jev 进行扩展实验。Jev 是一个面向状态评价的决策模型；在本实验中，它用于估计吃完所有剩余金币的代价，并作为 A* 搜索的启发式函数。该扩展不会替代 Q6 的基础实现，也不改变自动评分器对原始 `coinHeuristic` 的评分。

项目提供了一个可直接选择的 Jev 启发式函数：

```
jevCoinsScoreHeuristic(state, problem)
```

该函数会将当前勇士位置、完整迷宫和剩余金币集合发送给 Jev。Jev 使用有序 `score` 准则估计剩余代价；相同的“勇士位置 + 剩余金币集合”会使用缓存，避免重复请求。

Q6 默认使用 OpenRouter 的标准库 HTTP 客户端，无需额外安装依赖。如需安装项目列出的可选 TypeSafe SDK 依赖，在 `pj1-search` 目录运行：

```
python -m pip install -r jev/requirements.txt
```

首先通过环境变量配置 OpenRouter API Key。

Linux：

```
export OPENROUTER_API_KEY="your-openrouter-key"
```

PowerShell：

```
$env:OPENROUTER_API_KEY = "your-openrouter-key"
```

然后在 `pj1-search` 目录运行以下命令：

```
python hero.py -l trickySearch -p SearchAgent -a  
fn=astar,prob=CoinSearchProblem,heuristic=jevCoinsScoreHeuristic
```

可以在 `jev/q6.py` 的课程实验配置区调整：

- `Q6_SCORE_PROMPT`：Jev 判断剩余代价时使用的任务说明；
- `Q6_SCORE_LEVELS`：有序的剩余代价评分等级；

- `Q6_SCORE_MAX_COST`: 将 `Score` 映射为启发值时使用的最大代价。

实验时可以比较传统启发式和 Jev 启发式的路径代价、扩展节点数、API 请求数和运行时间。

注意: Jev 返回的启发值不保证可采纳或一致, 因此 A* 搜索得到的路径可能不是最优路径。底层 API、缓存、限速、重试和统计功能的说明见 `jev/README.md`。

Q7. α - β 剪枝

需要完成: `multiAgents.py` 中的 `AlphaBetaAgent.getAction` 方法。

最终目标: 实现支持多条恶龙的 α - β 剪枝搜索, 在保持极小极大结果的同时减少节点扩展。

让我们重新回到对抗搜索。本题需要编写一个使用 α - β 剪枝来更高效地探索极小极大树的新智能体 `AlphaBetaAgent`。其中一个挑战是将 α - β 剪枝的逻辑适当地扩展到多个最小化智能体。

通过执行下方命令并对比两种算法在相同迷宫中的表现, 可以明显感受到搜索速度的提升(深度为 3 的 α - β 剪枝运行速度可能与深度为 2 的极小极大算法相当)。理想情况下, 在 `smallClassic` 布局上, 深度为 3 的 α - β 剪枝每步只需运行几秒钟, 甚至更快。

```
python hero.py -p AlphaBetaAgent -a depth=3 -l smallClassic
```

`AlphaBetaAgent` 的极小极大值应该与 `MinimaxAgent` 的极小极大值相同, 尽管由于不同的平局处理方式, 它选择的动作可能会有所不同。同样, `minimaxClassic` 布局中初始状态的极小极大值分别为 9、8、7 和 -492 (对应深度 1、2、3 和 4)。

小贴士:

1. 由于评分基于智能体在搜索时是否探索了正确数量的状态, 因此不要对子节点进行重新排序。换句话说, 后继状态应始终按照 `GameState.getLegalActions` 返回的顺序进行处理。
2. 为了与自动评分器保持一致, 请不要在相等的情况下进行剪枝。
3. 正确实现的 α - β 剪枝会导致勇士在某些测试中输掉游戏——这是正常现象。

以下伪代码代表了此问题对应的算法:

Alpha-Beta Implementation

α : MAX's best option on path to root
 β : MIN's best option on path to root

```
def max-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = -\infty$   
    for each successor of state:  
         $v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v > \beta$  return  $v$   
         $\alpha = \max(\alpha, v)$   
    return  $v$ 
```

```
def min-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = +\infty$   
    for each successor of state:  
         $v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v < \alpha$  return  $v$   
         $\beta = \min(\beta, v)$   
    return  $v$ 
```

成绩评测:

通过运行以下命令进行评分:

```
python autograder.py -q q7
```

如果想在没有图形界面的情况下运行，使用：

```
python autograder.py -q q7 --no-graphics
```

Q8. 期望极大化

需要完成： `multiAgents.py` 中的 `ExpectimaxAgent.getAction` 方法。

最终目标： 将恶龙建模为均匀随机行动的概率节点，计算期望值并返回 Hero 的最佳动作。

极小极大算法和 α - β 剪枝算法都假设智能体在与一个做出最优决策的对手对战。然而，实际情况并非总是如此。在这个问题中，需要实现一个 `ExpectimaxAgent`，它需要能够模拟可能做出次优选择的概率性行为智能体。

由于算法具有通用性，可以运行以下命令测试算法在一些通用测试案例上的表现：

```
python autograder.py -q q8
```

建议先在这些小测试案例上调试，以便快速发现 bug。

完成后，可以在 Hero 游戏中观察其表现。随机恶龙不是最优的极小极大智能体，因此用极小极大搜索来建模它们可能并不合适。`ExpectimaxAgent` 不再对所有恶龙动作取最小值，而是根据智能体对恶龙行为的估计取期望值。为简化代码，假设对手只会在 `getLegalActions` 返回的动作中均匀随机选择。

要查看 `ExpectimaxAgent` 在 Hero 游戏中的表现，运行：

```
python hero.py -p ExpectimaxAgent -l minimaxClassic -a depth=3
```

可以观察到勇士在恶龙近身时表现得更大胆。但对于以下两个场景的结果：

```
python hero.py -p AlphaBetaAgent -l trappedClassic -a depth=3 -q -n 10
python hero.py -p ExpectimaxAgent -l trappedClassic -a depth=3 -q -n 10
```

`ExpectimaxAgent` 大约会赢得一半的游戏，而 `AlphaBetaAgent` 则总是输掉比赛。请思考为什么会出现不同的结果。

成绩评测：

通过运行以下命令进行评分：

```
python autograder.py -q q8
```

Q9. 改进评价函数

需要完成： `multiAgents.py` 中的 `betterEvaluationFunction` 函数。

最终目标： 设计一个评价当前 Hero 状态的启发式函数，在深度 2 搜索下取得较好的胜率、得分和运行效率。

请在提供的 `betterEvaluationFunction` 函数中为 Hero 编写一个更好的状态评价函数。该评价函数应评估当前游戏状态，而不是单次动作。你可以根据金币数量与距离、恶龙位置、剑和弱化状态等游戏特征，设计能够指导搜索决策的启发式评价函数。

通过深度为 2 的搜索，合格的评价函数应该能够在 `smallClassic` 布局中，在与一个随机恶龙对战时赢得超过一半次数的游戏，并保持合理的运行速度。

本题按下面的基础评分标准评测，完成本题不需要调用 `Jev`。后面的 `Jev` 实验仅供感兴趣的同学探索，**不计入成绩，也不要求完成或提交**。

评分标准：

自动评分器将在 `smallClassic` 布局上运行智能体 10 次。

1. 胜利次数

- 至少赢得 1 次游戏：+1 分；
- 至少赢得 5 次游戏：+2 分；
- 赢得全部 10 次游戏：+3 分。

2. 平均得分

- 平均得分至少达到 500 分（包括失败游戏的分数）：+1 分；
- 平均得分至少达到 1000 分：+2 分。

3. 运行效率

- Hero 平均每局累计决策时间少于 30 秒，且 10 局均未触发超时：+1 分。

注意：

- 只有至少赢得 5 场游戏后，才能获得平均得分和运行时间相关的额外分数。
- 单局 120 秒限制用于防止程序长时间无响应，不等同于平均运行时间评分标准。
- 上述评分针对基础 Q9；可选 `Jev` 扩展不适用这些胜利次数与运行时间要求。

成绩评测：

通过运行以下命令进行评分：

```
python autograder.py -q q9
```

如果想在没有图形界面的情况下运行，使用：

```
python autograder.py -q q9 --no-graphics
```

可选扩展：使用 `Jev` 辅助状态评价（不计入成绩）

与 Q6 中使用 `Jev` 估计剩余路径代价不同，本实验将 `Jev` 的整体局面评分作为传统状态评价函数的额外特征，观察它对 Expectimax 搜索决策和游戏结果的影响。`Jev` 不替代 Expectimax，也不要求替代自己设计的传统评价函数。

`Jev` 接口返回一个整体状态评分 `jev_score`，范围为 0-100，越高表示模型判断当前局面越有利。该分数是相对的局面质量信号，不代表获胜概率。输入包括地图、Hero 位置、剩余金币与剑、恶龙位置、方向及弱化计时等，但不包含当前累计游戏分数；传统评价部分仍可使用游戏分数。

示例中的 `state` 是待评价的 `GameState`。同一个 `evaluator` 应在多次评价之间复用，以保留缓存：

```
from jev.q9 import Q9JevEvaluator

jev = Q9JevEvaluator()
jev_score = jev.score(state)

# 将 jev_score 作为传统评价函数的一个可选特征。
jev.close() # 所有评价结束后关闭。
```

建议先进行一组简单的配对实验：使用 `smallClassic`、一个随机恶龙和深度为 2 的 Expectimax，分别运行一局传统评价和一局 Jev 混合评价，并使用相同的随机种子。可以记录胜负、最终得分、行动步数、API 请求数和运行时间。

在 `pj1-search` 目录运行：

```
python -u -m jev.q9
```

默认运行一局传统对照和一局真实 Jev，随机种子为 0。程序会加载项目 `.env` 中的 `api_key`，或读取环境变量 `OPENROUTER_API_KEY`。如果希望限制实验时间或请求次数，可以运行：

```
python -u -m jev.q9 --max-seconds 600 --max-requests 300
```

网络速度、服务负载和请求数量都会影响运行时间，并可能产生 API 费用。单局结果具有偶然性，不能据此证明某种评价方法一定更好；实验中途停止、API 失败或未完成整局均不影响成绩。完整的配置、结果记录和失败处理说明见 `jev/README.md` 的 Q9 小节。

附

本项目包含一个自动评分工具，可以用它来进行评分。运行以下命令，为所有问题评分：

```
python autograder.py
```

通过以下命令只运行特定问题的评分，例如 `q2`：

```
python autograder.py -q q2
```

参数 `-q` 控制待评估的题目。